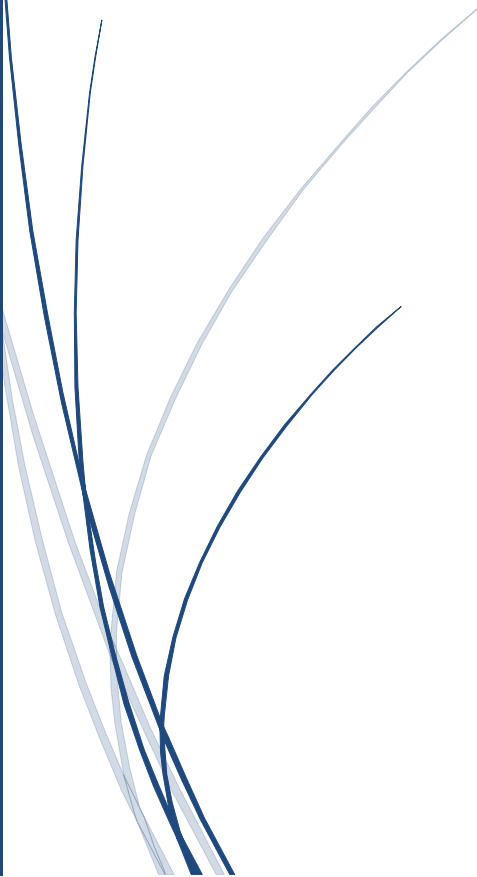




11-9-2026

**ACTIVIDAD – COMPRENDER Y
COMPLETAR CÓDIGO EN JAVASCRIPT**
POO: Animal, Propietario y Atención



Sofia Alejandra Trujillo Ordoñez
ADSO - 3407734

Contenido

1. Lea la situación	3
2. Comprendamos primero el Código	3
3. Comprender this	4
4. Comprender el objeto	4
5. Comprender el método	5
Objeto que ejecuta el método	5
Método ejecutado	5
¿Qué información muestra?	5
6. Ahora observe Propietario	5
¿Qué significa this.mascota = []?	5
7. ¿Por que usamos []?	6
8. Comprender push()	6
push() sirve para	6
9. Seguir los datos	6
¿Qué objeto quedó guardado?	7
10. ¿Cómo llegamos al nombre?	7
11. Agregar otra mascota	7
12. Acceder a Michi	7
13. Completar los métodos del arreglo	7
14. Ahora aparece Atención	8
15. Crear una atención	8
16. Una mascota puede tener varias atenciones	8
¿Por qué atenciones debe ser un arreglo?	9
17. Registrar una atención	9
18. Comprender la ruta completa	9
19. Complete	9
Nombre de Colmillo	9

Raza de Colmillo	9
Diagnóstico.....	9
Tratamiento:	9
20. Actividad final – completar el programa.....	10
21. Crear los objetos	11
22. Reflexión	12
1. ¿Por qué mascotas es un arreglo?	12
2. ¿Por qué atenciones es un arreglo?	12
3. ¿Qué significa [0]?.....	12
4. ¿Qué significa .nombre?.....	12
5. Explique qué hace alexa.mascotas[0].nombre:.....	12
6. Explique qué hace alexa.mascotas[0].atenciones[0].diagnostico:	12
Resultado esperado	12

1. Lea la situación

Una veterinaria necesita desarrollar un sistema para registrar mascotas y sus propietarios. Un animal tiene nombre, raza, especie y edad. Un propietario tiene nombre, dirección, teléfono y una lista de mascotas. Una mascota puede tener varias atenciones veterinarias. Una atención tiene fecha, motivo, diagnóstico y tratamiento.

2. Comprendamos primero el Código

```
class Animal {  
  
    nombre  
    raza  
    especie  
    edad  
  
    constructor(nombre, raza, especie, edad) {  
  
        this.nombre = nombre  
        this.raza = raza  
        this.especie = especie  
        this.edad = edad  
  
    }  
  
    registrarNombre(nuevoNombre) {  
        this.nombre = nuevoNombre  
    }  
  
    registrarRaza(nuevaRaza) {  
        this.raza = nuevaRaza  
    }  
  
    registrarEspecie(nuevaEspecie) {  
        this.especie = nuevaEspecie  
    }  
  
    registrarEdad(nuevaEdad) {  
        this.edad = nuevaEdad  
    }  
  
    mostrarInformacion() {  
        console.log(this.nombre + " " + this.raza)  
    }  
}
```

2.1. ¿Cuál es el nombre de la clase? Animal.

2.2. ¿Cuáles son sus atributos? nombre, raza, especie y edad.

2.3. ¿Para qué sirve el constructor? Sirve para crear e inicializar un objeto con sus datos.

2.4. ¿Qué hace registrarNombre()? Sirve para cambiar o registrar el nombre del animal.

2.5. ¿Qué hace mostrarInformacion()? Sirve para mostrar en consola el nombre y la raza del animal.

3. Comprender this

Observe: `this.nombre = nombre`

- **this hace referencia a:** El objeto actual.
- **this.nombre representa:** El atributo nombre del objeto.
- **nombre representa:** El valor que recibe el constructor o método como parámetro.

4. Comprender el objeto

```
let colmillo = new Animal(  
  "colmillo",  
  "pitbull",  
  "perro",  
  12  
)
```

Nombre: colmillo

Raza: pitbull

Especie: perro

Edad: 12

colmillo es:

colmillo es: ☐ Una clase ☒ Un objeto ☐ Un método ☐ Un arreglo

5. Comprender el método

Observe: colmillo.mostrarInformacion()

Objeto que ejecuta el método: colmillo

Método ejecutado: mostrarInformacion()

¿Qué información muestra? Muestra el nombre y la raza de Colmillo.

6. Ahora observe Propietario

```
class Propietario {  
  
    nombrePropietario  
    Direccion  
    telefono  
  
    constructor(nombre, direccion, telefono) {  
  
        this.nombrePropietario = nombre  
        this.direccion = direccion  
        this.telefono = telefono  
  
        this.mascota = []  
    }  
}
```

nombrePropietario → nombre del propietario

Direccion → dirección del propietario

telefono → teléfono del propietario

mascota → lista de mascotas

¿Qué significa this.mascota = []?

Significa que se crea un arreglo vacío para guardar las mascotas del propietario.

7. ¿Por que usamos []?

El propietario puede tener varias mascotas. El arreglo permite almacenar varios objetos mascota.

mascota es un arreglo porque puede almacenar varios elementos.

8. Comprender push()

```
registrarMascota(nuevaMascota) {  
    this.mascota.push(nuevaMascota)  
}
```

push() sirve para: Agregar un elemento al final.

- ☐ Eliminar el primer elemento.
- ☒ Agregar un elemento al final.
- ☐ Agregar un elemento al inicio.
- ☐ Eliminar el último elemento.

9. Seguir los datos

```
let alexa = new Propietario(  
    "Alexa",  
    "123",  
    "1111"  
)
```

```
let colmillo = new Animal(  
    "colmillo",  
    "pitbull",  
    "perro",  
    12  
)
```

```
alexa.registrarMascota(colmillo)
```

Antes: alexa.mascota = []

Después: alexa.mascota = [colmillo]

¿Qué objeto quedó guardado? El objeto colmillo.

10. ¿Cómo llegamos al nombre?

alexa → mascotas → [0] → nombre

Complete:

alexa.mascota[0].nombre

¿Qué resultado obtenemos? colmillo

11. Agregar otra mascota

```
let michi = new Animal(  
  "Michi",  
  "Criollo",  
  "Gato",  
  3  
)
```

alexa.registrarMascota(michi)

Ahora el arreglo contiene:

alexa.mascota = [colmillo, michi]

12. Acceder a Michi

Si Colmillo está en alexa.mascota[0], ¿en qué posición está Michi?

Posición de Michi: alexa.mascota[1]

Código: console.log(alexa.mascota[1].nombre)

Resultado: Michi

13. Completar los métodos del arreglo

Banco de palabras: push() pop() unshift() shift()

Agregar al final: this.mascotas.push(mascota)

Eliminar del final: this.mascotas.pop()

Agregar al inicio: this.mascotas.unshift(mascota)

Eliminar del inicio: this.mascotas.shift()

14. Ahora aparece Atención

```
class Atencion {
  constructor(fecha, motivo, diagnostico, tratamiento) {
    this.fecha = fecha
    this.motivo = motivo
    this.diagnostico = diagnostico
    this.tratamiento = tratamiento
  }

  mostrarInformacion() {
    console.log("Fecha: " + this.fecha)
    console.log("Motivo: " + this.motivo)
    console.log("Diagnóstico: " + this.diagnostico)
    console.log("Tratamiento: " + this.tratamiento)
  }
}
```

15. Crear una atención

```
let atencion1 = new Atencion(
  "11/09/2026",
  "Dolor abdominal",
  "Gastritis",
  "Dieta especial"
)
```

Complete: atencion1.mostrarInformacion()

16. Una mascota puede tener varias atenciones

```
constructor(nombre, raza, especie, edad) {

  this.nombre = nombre
  this.raza = raza
  this.especie = especie
  this.edad = edad
}
```

```
    this.atenciones = []  
  }
```

¿Por qué atenciones debe ser un arreglo?

Porque una mascota puede tener varias atenciones veterinarias y el arreglo permite guardarlas.

17. Registrar una atención

```
registrarAtencion(nuevaAtencion) {  
    this.atenciones.push(nuevaAtencion)  
}
```

Después: colmillo.registrarAtencion(atencion1)

18. Comprender la ruta completa

alex → propietario

.mascotas → arreglo de mascotas

[0] → primera mascota

.atenciones → arreglo de atenciones

[0] → primera atención

.diagnostico → diagnóstico de la atención

19. Complete

Nombre de Colmillo: console.log(alex.mascotas[0].nombre)

Raza de Colmillo: console.log(alex.mascotas[0].raza)

Diagnóstico: console.log(alex.mascotas[0].atenciones[0].diagnostico)

Tratamiento: console.log(alex.mascotas[0].atenciones[0].tratamiento)

20. Actividad final – completar el programa

```
class Animal {
    constructor(nombre, raza, especie, edad) {
        this.nombre = nombre
        this.raza = raza
        this.especie = especie
        this.edad = edad
        this.atenciones = []
    }

    registrarAtencion(nuevaAtencion) {
        this.atenciones.push(nuevaAtencion)
    }

    mostrarInformacion() {
        console.log(
            this.nombre + " " +
            this.raza + " " +
            this.especie + " " +
            this.edad
        )
    }
}

class Propietario {
    constructor(nombre, direccion, telefono) {
        this.nombre = nombre
        this.direccion = direccion
        this.telefono = telefono
        this.mascotas = []
    }

    registrarMascota(nuevaMascota) {
        this.mascotas.push(nuevaMascota)
    }

    mostrarInformacion() {
        console.log(
            "Mascota: " +
            this.mascotas[0].nombre +
            " Propietario: " +
            this.nombre
        )
    }
}
```

```

}

class Atencion {
  constructor(fecha, motivo, diagnostico, tratamiento) {
    this.fecha = fecha
    this.motivo = motivo
    this.diagnostico = diagnostico
    this.tratamiento = tratamiento
  }

  mostrarInformacion() {
    console.log("Fecha: " + this.fecha)
    console.log("Motivo: " + this.motivo)
    console.log("Diagnóstico: " + this.diagnostico)
    console.log("Tratamiento: " + this.tratamiento)
  }
}

```

21. Crear los objetos

```

let colmillo = new Animal(
  "Colmillo",
  "Pitbull",
  "Perro",
  12
)

let alexa = new Propietario(
  "Alexa",
  "Calle 10",
  "3001234567"
)

let atencion1 = new Atencion(
  "11/09/2026",
  "Dolor abdominal",
  "Gastritis",
  "Dieta especial"
)

alexa.registrarMascota(colmillo)
colmillo.registrarAtencion(atencion1)
colmillo.mostrarInformacion()

```

alexa.mostrarInformacion()
atencion1.mostrarInformacion()

22. Reflexión

1. ¿Por qué mascotas es un arreglo?

Porque un propietario puede tener varias mascotas, por eso se utiliza un arreglo para almacenarlas.

2. ¿Por qué atenciones es un arreglo?

Porque una mascota puede tener varias atenciones veterinarias.

3. ¿Qué significa [0]?

Significa que estamos accediendo al primer elemento de un arreglo.

4. ¿Qué significa .nombre?

Significa que estamos accediendo al atributo nombre de un objeto.

5. Explique qué hace alexa.mascotas[0].nombre:

Accede al arreglo de mascotas de Alexa, selecciona la primera mascota y obtiene su nombre. El resultado es Colmillo.

6. Explique qué hace alexa.mascotas[0].atenciones[0].diagnostico:

Accede a las mascotas de Alexa, selecciona la primera mascota, entra a sus atenciones, selecciona la primera atención y obtiene su diagnóstico. El resultado es Gastritis.

Resultado esperado

La lógica del programa es: Propietario → mascotas[] → mascota[0] → atenciones[] → atención[0] → diagnóstico. Un propietario puede tener varias mascotas, una mascota puede tener varias atenciones y cada atención contiene información como fecha, motivo, diagnóstico y tratamiento.